



BEYOND X TECHNICAL PAPER

TP-010 V6

Large-Scale Oracle 19c on Exadata to Amazon Aurora MySQL 8.0 Modernization

Engineering a ~50 TB Mission-Critical Database Migration
While Preserving 35+ Years of Business Capability



BeyondX LLC

Technology with Purpose. Opportunity with Impact.
Imagination to Implementation.

Version 6.0 | September 2026

Executive Summary

Modernizing approximately 50 TB of Oracle 19c data from an Oracle Exadata environment to Amazon Aurora MySQL 8.0 was not a database-copy exercise. The source platform had evolved since the 1990s and embodied more than three and a half decades of application development, database engineering, operational refinement, and institutional knowledge. The central challenge was therefore not merely to move data, but to ensure that required business functionality and operational capability survived a change of database engine and performance architecture.

AWS Schema Conversion Tool (AWS SCT) accelerated assessment and generated an initial target-model skeleton, but production readiness required extensive manual engineering. The team reviewed tables, keys, partitioning, functional indexes, constraints, triggers, procedures, jobs, archival logic, nested structures, application dependencies, performance behavior, and fallback requirements. AWS Database Migration Service (AWS DMS) provided Full Load + CDC for forward migration and reverse CDC during the controlled fallback window.

CORE POSITION	At ~50 TB scale, migration success depends on preserving business capability, re-engineering target behavior, and proving the result with production-representative evidence.
----------------------	--

What V6 adds

- **35+ years of capability preservation:** manual analysis was explicitly used to ensure that functionality embedded since the 1990s was not lost.
- **Oracle Exadata to Aurora performance transition:** tuning moved from an Oracle/Exadata ecosystem to workload-driven Aurora MySQL engineering.
- **Production-simulated performance testing:** high-CPU SQL and long-running transactions were captured, analyzed, and used to design composite indexes.
- **Aurora capacity and parameter engineering:** the target used db.r8g.24xlarge with deliberate parameter tuning for concurrency, observability, temporary-table capacity, and CDC.
- **Reverse CDC prerequisites:** Aurora binary logging and retention were engineered so Aurora could act as the source during fallback.
- **Index lifecycle governance:** redundant and potentially unused indexes are reviewed using MySQL sys schema views and representative business-cycle evidence.
- **Granular AI-assisted engineering:** ChatGPT and AI agents are mapped across discovery, conversion, DMS, validation, tuning, cutover, troubleshooting, documentation, and decommission readiness.
- **Measured outcome:** the migration completed with negligible issues; after approximately two months of stable target operation, the legacy Oracle source was decommissioned.

1. Modernization Context: Preserve Capability, Change Implementation

The Oracle platform had been under development since the 1990s. Over more than 35 years, business behavior accumulated not only in application code but also in database structures and operational practices: tables, relationships, constraints, indexes, function-based indexes, triggers, stored logic, jobs, partitioning, archival processes, data structures, and exception handling.

That history made manual analysis a risk-control discipline. The objective was not to recreate every Oracle object literally. It was to identify the business or operational capability represented by each significant construct and ensure that an equivalent capability existed after modernization - even when Aurora MySQL required a fundamentally different implementation.

BEYONDX PRINCIPLE

Modernize the implementation. Preserve the capability. A legacy database represents accumulated business behavior, not merely DDL and data.

1.1 Scale changes the problem

At approximately 50 TB, design had to account for source read pressure, target write capacity, network throughput, CDC backlog, LOB behavior, restart scope, validation windows, cutover duration, and recovery. Very large and LOB-heavy tables became independent engineering workstreams because a single failed long-running load could materially affect the schedule.

1.2 Modernization objectives

- Preserve application-visible business semantics and data integrity.
- Create consistent Aurora MySQL DEV, PreProd, and PROD target structures.
- Use lower-environment migrations as prerequisite evidence for production approval.
- Design restartable DMS workloads, observable operations, and an executable fallback path.
- Re-establish application performance on Aurora MySQL through target-specific tuning rather than inherited source assumptions.

2. Discovery, Source Drift, and the Production Baseline

Discovery extended beyond tables and columns to the behavior of the system. Long-standing structural drift across the Oracle source environments was a major complication: DEV and PreProd had not been synchronized with PROD for decades. Yet successful Oracle DEV to Aurora MySQL DEV and Oracle PreProd to Aurora MySQL PreProd migrations, with zero unresolved migration issues, were required as prerequisite signoffs before production migration.

Discovery area	Representative scope
Data structures	Tables, partitions, nested structures, LOBs, very large tables
Integrity	PKs, unique keys, FKs, check constraints, trigger-enforced rules
Access paths	B-tree/composite indexes, function-based indexes, partition pruning
Programmability	Triggers, procedures, functions, sequences, jobs, application dependencies
Operations	Archive/purge, maintenance, monitoring, restart, recovery, cutover
Performance	High-CPU SQL, long transactions, access paths, target concurrency

2.1 Production-derived DMS table mappings

Production metadata became the authoritative migration baseline. Detailed AWS DMS table-mapping files explicitly identified required tables and included columns corresponding to the production definition. This was intensive work because lower environments could differ from production, but it allowed migration logic to be rehearsed against the production-intended schema before production cutover.

ENGINEERING LESSON	When lower environments cannot be trusted as structural baselines, production metadata must become the migration source of truth.
---------------------------	--

3. AWS SCT as Skeleton; Manual Analysis as Capability Assurance

AWS SCT accelerated assessment and generated an initial Aurora MySQL target-model skeleton. The output was deliberately treated as a starting point, not production-ready DDL. The age and complexity of the Oracle system meant that automated conversion could not prove functional equivalence.

Manual analysis reviewed each important construct in terms of the capability it protected. Where a direct target equivalent did not exist, behavior was deliberately re-engineered through Aurora MySQL constructs, triggers, procedures, Java application processes, application controls, or other validated mechanisms.

Review area	Question asked during modernization
Tables / datatypes	Does the target preserve data meaning, scale, nullability, and application expectations?
Keys / relationships	What business invariant is being protected, and how will it be enforced on the target?
Indexes	What query behavior or uniqueness rule does this access path support?
Triggers / procedures	What business behavior, validation, audit, or integrity logic is embedded here?
Jobs / archives	Is this application orchestration or database maintenance responsibility?
Dependencies	Which applications, reports, interfaces, and operational processes depend on the object?
ENGINEERING LESSON	Schema conversion is not schema modernization. Automated tools accelerate discovery; accountable engineering preserves capability.

3.1 Nested tables were flattened before migration

Oracle nested-table structures were flattened and remodeled in Oracle before database migration. Application and database dependencies were adjusted and validated against the flattened representation first. AWS DMS therefore migrated an already-modernized relational representation; DMS was not used as the nested-table transformation mechanism.

4. Partitioning, Keys, Referential Integrity, and Trigger Redesign

The Oracle source had only 12 partitioned tables. In Aurora MySQL, additional high-volume tables were partitioned to improve manageability, retention, archival operations, and workload performance. The target partitioning strategy was therefore intentionally broader than the source.

For a generalized example, consider a table partitioned monthly by CRTE_DT whose Oracle primary key is APLCTN_SEQ_NBR. In the Aurora MySQL partitioned design, unique keys must include the partitioning column. The target key therefore becomes conceptually (APLCTN_SEQ_NBR, CRTE_DT). That composite key protects uniqueness of the pair, but a business requirement for global APLCTN_SEQ_NBR uniqueness requires separate enforcement.

Oracle source	Aurora MySQL target	Business implication
Monthly CRTE_DT partition; PK APLCTN_SEQ_NBR	Monthly CRTE_DT partition; PK includes APLCTN_SEQ_NBR + CRTE_DT	Global sequence uniqueness requires separate control
12 partitioned tables	Additional high-volume tables partitioned	Target designed for target scale and operations
Declarative relationships	Affected partitioned-table relationships re-engineered	Triggers, procedures, control tables, or application validation

4.1 Trigger decomposition

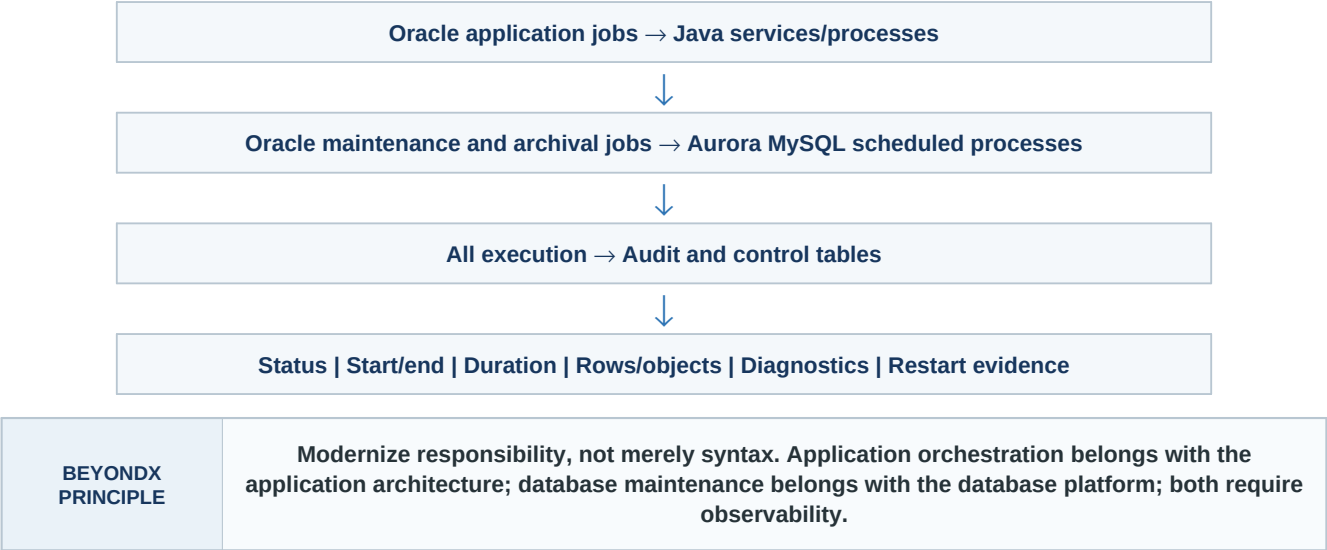
Oracle can combine multiple DML events in a single trigger, such as BEFORE INSERT OR UPDATE OR DELETE, and branch internally by event. MySQL triggers are defined for one timing and one event at a time. Equivalent behavior therefore required separate INSERT, UPDATE, and DELETE triggers where appropriate, together with review of shared logic, execution order, recursion risk, errors, and DMS interactions.

BEYONDx PRINCIPLE	Preserve the business invariant, not necessarily the source implementation. Equivalent behavior does not always mean equivalent database objects.
--------------------------	--

5. Jobs, Scheduling, and Operational Modernization

Application-related Oracle jobs were converted to Java processes. Database-centric maintenance and archival jobs remained database responsibilities and were re-engineered as Aurora MySQL scheduled processes. This separated application orchestration from database housekeeping while reducing Oracle-specific coupling.

Operational audit/control tables captured process name, start time, completion time, duration, success/failure status, rows or objects processed, checkpoints, and diagnostic/error information. These records supported monitoring, troubleshooting, restart decisions, support, and evidence of successful execution.



6. AWS DMS Architecture at ~50 TB Scale

AWS DMS provided the primary data-movement mechanism. Full Load established the target baseline while CDC maintained convergence as source changes continued. Endpoints, replication resources, task mappings, LOB settings, table preparation, logging, validation, and restart behavior were treated as an orchestrated migration subsystem.

A monolithic task would have concentrated risk. Workloads were decomposed by table size, change rate, LOB profile, partition strategy, and operational criticality. Heavy tasks were introduced in controlled waves so source and target pressure could be observed before additional workloads were started.

6.1 Very large tables, LOBs, and parallel loading

Several very large tables contained substantial LOB data. Some initial DMS loads failed after approximately 25 hours. Repeating the same monolithic pattern would have created unacceptable schedule and recovery risk. DMS parallel-load techniques and more manageable load units were therefore used to improve throughput, restartability, fault isolation, and operational control.

ENGINEERING LESSON	At very large scale, migration architecture must include failure architecture: design how work fails, restarts, reconciles, and continues.
-------------------------------	---

6.2 Observability

- CDC source and target latency; task errors, retries, and table-level load state.
- Replication memory/swap pressure, source/target throughput, CPU, I/O, storage headroom, and query latency.
- Audit/control-table status, elapsed time, checkpoints, and diagnostics.

7. Reverse CDC and Fallback Engineering

AWS DMS was also used for reverse CDC during the controlled cutover period so that an unforeseen production issue could trigger a return to Oracle without discarding target-side changes. Reverse CDC was a temporary safety architecture, not the steady-state operating model.

Forward LOB movement mapped Oracle CLOB data to Aurora MySQL LONGTEXT. The reverse path required Aurora MySQL LONGTEXT changes to be applied back to Oracle CLOB columns. In the implemented approach, reverse LOB handling used Limited LOB mode with sizing engineered for the maximum required payload. Forward and reverse LOB paths were therefore validated independently rather than assumed to be configuration-symmetric.

7.1 Preparing Aurora MySQL to act as a CDC source

Fallback CDC required Aurora MySQL binary logging to be configured so AWS DMS could read row-level changes. The cluster parameter group was engineered for row-based logging with complete row images, and binary-log retention was set to cover the controlled fallback window and plausible interruption/recovery time. Any optional enhanced-binlog capability was treated as a performance optimization, not as a substitute for the required CDC settings.

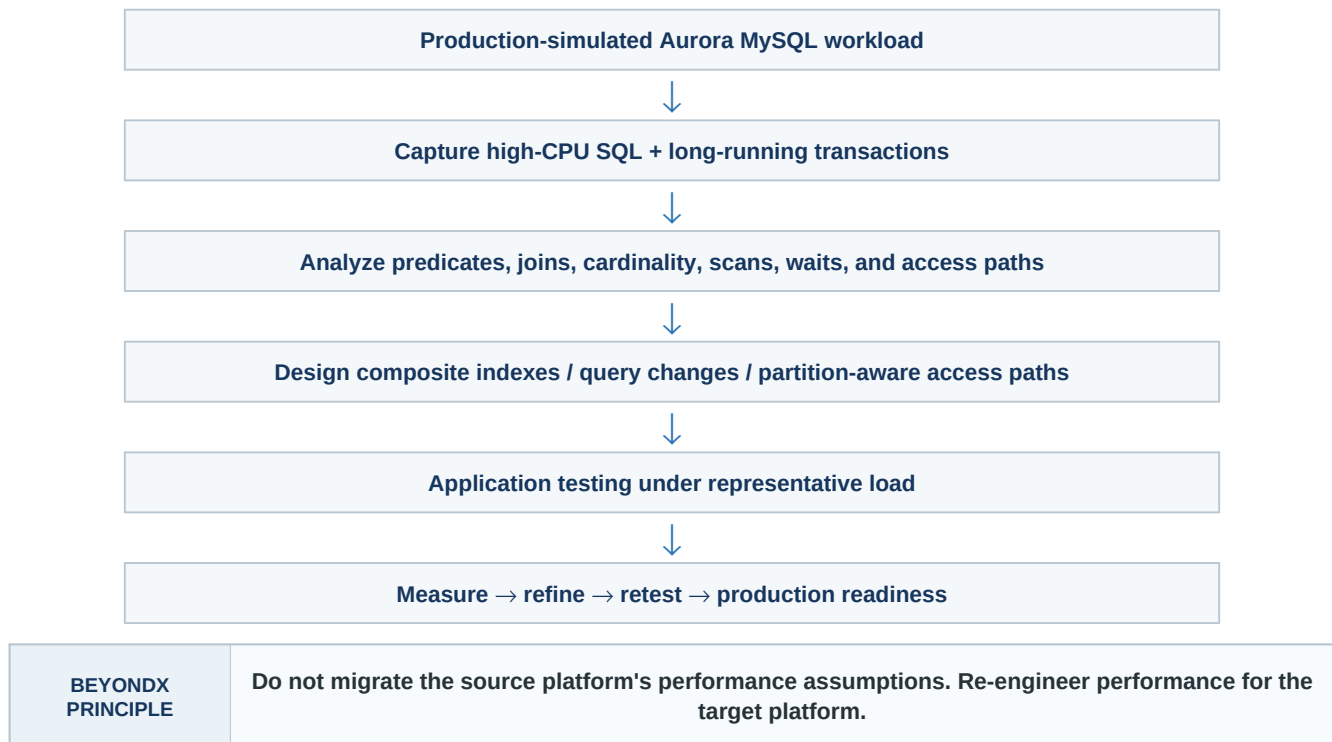
Area	Engineering intent
binlog_format = ROW	Capture row-level changes for CDC
binlog_row_image = FULL	Provide complete row images for update/delete processing
Binary-log retention	Protect CDC position through planned fallback window and temporary interruption
Reverse LOB settings	Support LONGTEXT-to-CLOB payload requirements
Validation	Prove reverse application, reconciliation, and rollback checkpoints

ENGINEERING LESSON	Rollback must be engineered, not merely documented. A fallback is credible only when the reverse data path is configured, retained, tested, and reconciled.
---------------------------	--

8. Performance Engineering: Oracle Exadata to Aurora MySQL

The source Oracle 19c database operated on Oracle Exadata, an environment with mature Oracle optimization and Exadata performance capabilities. Moving to Aurora MySQL changed the performance model. Source-platform assumptions could not simply be carried forward; application performance had to be re-established on the target platform.

In a production-simulated environment, the team captured high-CPU SQL activity, long-running transactions, predicates, joins, and execution characteristics. Candidate composite indexes were then designed from observed workload behavior rather than mechanically copying every Oracle index. Each change was subjected to application testing and measurement before acceptance.



8.1 Functional indexes

Oracle function-based indexes were inventoried and reviewed individually. Their purpose - filtering, normalization, expression-based lookup, uniqueness support, or query acceleration - was understood before selecting an Aurora MySQL-compatible implementation. The target design emphasized equivalent access behavior, not mechanical syntax conversion.

9. Aurora MySQL Capacity and Parameter Engineering

The target used the db.r8g.24xLarge instance class. Capacity selection was only the starting point. AWS documentation, technical publications, industry guidance, observed database behavior, and AI-assisted analysis were used as engineering inputs; generic tuning blueprints were not adopted wholesale. Parameter changes were calculated and validated against the real workload and production-simulated evidence.

Engineering area	Engineering approach
Concurrency / parallelism	Parameters and DMS task concurrency tuned against measured source/target pressure
Slow-query capture	Slow and expensive SQL captured for evidence-driven tuning
Temporary tables	Temporary-table capacity increased from defaults where representative workloads required it
CDC / binlogs	Cluster-level binary logging and retention engineered for reverse CDC fallback
Transactions	Long-running transactions and lock/deadlock behavior analyzed under target semantics
Indexes	Composite indexes designed from observed Aurora workload and application tests
Change control	Parameter and index changes validated in simulated production conditions before acceptance

ChatGPT assisted with calculations and reasoning about memory, concurrency, parameter interactions, and the expected effect of increases from defaults. Candidate changes were then tested against the actual environment. The tuning cycle connected infrastructure, database parameters, SQL behavior, and application testing. A larger instance did not remove the need to identify inefficient access paths or long-running transactions; conversely, query tuning had to be interpreted in the context of memory, temporary-table behavior, concurrency, and migration load.

BEYONDx PRINCIPLE	Cloud database defaults are starting points, not production performance designs. Capacity, parameters, SQL, indexes, and application behavior must be engineered and validated together.
------------------------------	---

10. Index Lifecycle Governance

Index engineering continued after cutover. Aurora MySQL/MySQL provides sys schema views that help identify redundant indexes and indexes not observed in use. These views provide evidence for review; they are not automatic deletion instructions.

View	Use in governance
sys.schema_redundant_indexes	Identify duplicate or overlapping indexes that may add storage and write-maintenance cost
sys.schema_unused_indexes	Identify indexes with no observed usage during the statistics observation period
sys.schema_index_statistics	Review index-level workload statistics as additional evidence

Potentially unused indexes must be evaluated across a representative business cycle. An index that appears unused during ordinary daily OLTP activity may support month-end, quarter-end, annual, archival, regulatory, or administrative processing. For that reason, the modernization strategy includes periodic analysis and a broader annual review before controlled retirement of an index.

The lifecycle is deliberate: create indexes from demonstrated workload requirements, validate their benefit, observe their utilization, review redundant or unused candidates, and retire only after business-cycle and application analysis.

BEYONDx PRINCIPLE	Indexes should have a lifecycle. Create from evidence, validate benefit, observe utilization, and retire only after representative business-cycle analysis.
------------------------------	--

11. Temporal Semantics and Transaction Behavior

Application requirements required both Oracle and Aurora MySQL to preserve the established EST-based temporal behavior. UTC conversion was not introduced as part of this migration. The governing requirement was preservation of application-visible time semantics.

Oracle DATE stores date and time to the second, whereas MySQL DATE is date-only. MySQL DATETIME and TIMESTAMP also have different timezone behavior. Type selection and migration rules were therefore validated against the business meaning of each column.

11.1 Isolation, locks, and deadlocks

Oracle and MySQL-compatible engines differ in concurrency implementation and default isolation behavior. Application transactions that performed acceptably on Oracle can exhibit different lock duration or deadlock patterns after migration. READ COMMITTED can be evaluated for OLTP workloads when REPEATABLE READ produces undesirable locking behavior and application consistency requirements permit the change; such decisions require transaction-level testing and are not universal tuning rules.

- Capture conflicting SQL and transaction sequence for deadlocks.
- Confirm selective indexes minimize scanned and locked rows.
- Standardize object access order where application changes are feasible.
- Keep transactions short and retry deadlock victims safely.
- Validate isolation changes against business consistency requirements.

12. Validation, Cutover, and Production Readiness

Validation began during discovery and continued through source prerequisite changes, target engineering, DEV/PreProd DMS tests, Full Load, CDC, performance testing, cutover, and stabilization. Lower-environment success was a production prerequisite, not an optional rehearsal.

Validation layer	Representative evidence
Structural	Tables, columns, datatypes, keys, index replacements, partitions, triggers, procedures
Volume / content	Row/partition counts, sampled comparisons, LOB completeness, nullability
CDC	Latency, applied changes, exceptions, convergence, binlog retention readiness
Application	Critical transactions, APIs, reports, Java jobs, error handling
Performance	High-CPU SQL, long transactions, composite indexes, concurrency, temporary tables
Operational	Aurora jobs, audit metrics, backups, monitoring, maintenance, fallback readiness

12.1 Cutover sequence

- Confirm target structural, data, performance, and lower-environment signoff evidence.
- Reduce or stop source writes according to the approved application plan and allow forward CDC to converge.
- Execute final reconciliation and application smoke tests, then redirect connectivity through controlled configuration.
- Monitor errors, latency, locks, DMS status, CPU, long transactions, and business transactions intensively.
- Maintain reverse CDC/fallback capability until acceptance criteria and rollback decision points are satisfied.

BEYONDx PRINCIPLE	Modernization is successful only when the legacy system can be retired with confidence.
------------------------------	--

13. AI-Assisted Engineering: BeyondX LLC and ChatGPT

BeyondX LLC used ChatGPT extensively as an AI-assisted engineering resource throughout the modernization effort. The objective was not to replace database engineers or client resources. It was to expand engineering coverage: accelerate repetitive analysis, challenge assumptions, identify omissions, generate validation evidence, and help experienced engineers reason through heterogeneous-platform differences.

Multiple AI tools can contribute to modernization. Effectiveness depends on the engineering context supplied to the agent, the quality of prompts and evidence, and disciplined human review. All production-affecting recommendations remained subject to architecture review, testing, security controls, change management, and observed database behavior.

AI OPERATING MODEL	AI accelerates the work - but, more importantly, it can challenge incomplete work. Human engineers remain accountable for correctness, testing, security, business behavior, and production decisions.
---------------------------	---

13.1 Granular AI opportunities - discovery through target design

Engineering activity	Where ChatGPT / AI agents can assist	Human accountability
Discovery inventory	Generate catalog queries; enumerate tables, views, LOBs, partitions, keys, constraints, indexes, triggers, procedures, jobs, sequences, types, nested structures, and dependencies.	Confirm completeness and access scope.
Metadata comparison	Compare DEV, PreProd, and PROD definitions; highlight missing objects/columns and structural drift.	Select the authoritative baseline.
Dependency analysis	Trace table-to-view-to-code-to-job relationships; build migration sequencing and impact questions.	Validate against actual applications and operations.
AWS SCT review	Interpret assessment findings; classify incompatibilities; compare generated DDL with source semantics.	Approve target implementation.
Datatype conversion	Analyze NUMBER precision/scale, DATE/TIMESTAMP, CLOB/BLOB, RAW, character semantics, nullability, and truncation risk.	Validate with real data and application semantics.
Nested structures	Identify ADTs/nested tables; propose flattening and parent/child mappings; generate dependency test cases.	Approve relational redesign and source prerequisite changes.
Partition/key design	Analyze partition keys, unique-key rules, pruning needs, and business invariants; propose target alternatives.	Approve integrity model.
FK/trigger redesign	Inventory relationships and multi-event Oracle triggers; propose target triggers, procedures, or application controls.	Validate behavior, recursion, errors, and execution order.
Index conversion	Identify function-based and composite indexes; infer purpose from predicates, joins, uniqueness, and workload.	Prove target access paths through testing.
Jobs and stored logic	Classify application orchestration vs. database maintenance/archive responsibilities; assist Java/SQL redesign.	Own architecture, code review, and schedules.

13.2 Granular AI opportunities - migration, validation, and performance

Engineering activity	Where ChatGPT / AI agents can assist	Human accountability
DMS mappings	Generate and review selection/transformation rules; compare include-column lists with production metadata.	Test every mapping and approve scope.
Task decomposition	Analyze table size, change rate, LOB profile, partitions, and failure cost; propose task groupings and parallel-load candidates.	Validate capacity and restart behavior.
LOB engineering	Generate max-LOB analysis SQL; compare LOB modes; reason about memory, truncation, throughput, and reverse CLOB/LONGTEXT handling.	Prove completeness with representative payloads.
CDC / reverse CDC	Review keys, binlog prerequisites, retention, latency, checkpoints, and fallback dependencies.	Prove forward and reverse paths before cutover.
Data reconciliation	Generate row counts, aggregates, min/max, null distributions, partition checks, hashes, and exception queries.	Define acceptance thresholds and investigate differences.
Functional testing	Turn discovered business rules and database behavior into positive, negative, boundary, and regression cases.	Application teams execute and sign off.
Performance analysis	Interpret slow SQL, high CPU, long transactions, plans, predicates, joins, scans, waits, locks, and deadlocks.	Reproduce findings and validate changes.
Parameter analysis	Model memory/concurrency implications; compare defaults and alternatives; calculate expected resource impact from measured conditions.	Change incrementally under controlled testing.
Composite indexes	Propose candidates and column order from actual predicates, joins, selectivity, and ordering.	Measure benefit and write cost.
Index governance	Query redundant/unused/index-statistics views; prepare evidence for periodic or annual review.	Retire only after representative business-cycle review.

13.3 Granular AI opportunities - cutover through retirement

Engineering activity	Where ChatGPT / AI agents can assist	Human accountability
Failure analysis	Challenge the plan with failure scenarios: task failure after many hours, CDC lag, LOB truncation, lock pile-up, storage pressure, or target regression.	Engineer and test recovery paths.
Cutover planning	Build runbooks, dependencies, timing sheets, decision points, go/no-go checklists, smoke tests, and evidence capture.	Cutover leadership owns execution.
Rollback planning	Build decision trees; identify routing, reverse CDC, reconciliation, and data-loss boundaries.	Business and technical owners authorize rollback.
Production monitoring	Correlate CPU, memory, storage, DMS latency, slow queries, long transactions, application errors, and audit tables.	Operations owns production response.
Incident troubleshooting	Correlate logs, SQL, metrics, recent changes, and symptoms; generate ranked diagnostic hypotheses and SQL checks.	Engineers verify every hypothesis.
Documentation	Continuously draft mappings, decisions, test evidence, procedures, troubleshooting guides, and knowledge-transfer material.	Technical owners review accuracy.
Decommission readiness	Generate residual dependency, connection, job, interface, reconciliation, backup, retention, and retirement checklists.	Organization authorizes source shutdown.

13.4 AI as a second engineering reviewer

A migration task reporting success is not sufficient evidence that the business capability survived. AI agents can be used as a persistent second reviewer that asks what may have been missed: Were all columns migrated? Were defaults and constraints preserved? What happened to function-based indexes? Did partitioning change uniqueness semantics? Are triggers behaviorally equivalent? Can CDC apply updates and deletes correctly? How will LOB completeness be proven? Which application or report depends on this object? What happens if the task fails after 25 hours?

This review pattern is especially valuable in long modernization programs where repetitive work can make omissions difficult to see. The goal is not to criticize individuals; it is to create a repeatable engineering control that improves coverage, consistency, traceability, and learning across the team.

BEYONDx AI PRINCIPLE	Use AI continuously to discover what may have been missed, generate validation evidence, examine failure scenarios, and challenge assumptions - while keeping production accountability with experienced engineers.
-----------------------------	--

14. Engineering Lessons Learned

- **Preserve capability, not just objects.** More than 35 years of embedded business behavior required manual semantic analysis.
- **Treat SCT output as a skeleton.** Automated conversion accelerates assessment; production architecture requires human review.
- **Let production metadata govern when lower environments drift.** Production-derived DMS mappings enabled meaningful rehearsals.
- **Decompose by risk and design for failure.** Large tables and LOBs need restartable, observable migration paths.
- **Engineer rollback.** Reverse CDC, binlog retention, and validated LOB handling make fallback executable.
- **Re-engineer target performance.** Exadata-era assumptions must be replaced with Aurora workload evidence, composite-index tuning, and application testing.
- **Use external guidance as input, not prescription.** Documentation, blogs, and AI suggestions are hypotheses until validated against the real environment.
- **Govern indexes throughout their lifecycle.** Redundant and unused-index analysis belongs in steady-state operations.
- **Use AI under human engineering control.** AI expands analysis capacity and challenges omissions; accountable engineers own correctness.
- **Measure completion by retirement.** A modernization proves itself when the target operates successfully and the legacy source can be decommissioned with confidence.

15. Reference Modernization Framework

Step	Discipline	Outcome
1	Discover	Inventory objects, dependencies, workload, environments, temporal semantics, operations, and embedded capability
2	Baseline	Use production metadata as authoritative source; reconcile lower-environment drift
3	Transform	Flatten source prerequisites; engineer Aurora-native schema, keys, partitions, integrity controls, jobs, and access paths
4	Rehearse	Execute DEV and PreProd migrations with production-intended DMS mappings and simulated production workload
5	Migrate	Run controlled Full Load + CDC with decomposed/parallel workloads
6	Validate	Reconcile structure, data, CDC, applications, performance, jobs, and operations
7	Cut over	Converge CDC, activate target, maintain reverse CDC, and monitor intensively
8	Stabilize	Tune workload, govern indexes, prove operations, and retire source after approval
9	Retire	After approximately two months of stable target operation, decommission the legacy Oracle source

Conclusion

A ~50 TB Oracle 19c on Exadata to Amazon Aurora MySQL 8.0 modernization is an enterprise engineering program, not a database copy. The successful path combined production-aware discovery, preservation of 35+ years of business capability, source prerequisite modernization, SCT-assisted skeleton generation, manual target redesign, production-derived DMS mappings, lower-environment rehearsals, specialized LOB and parallel-load engineering, partition and integrity redesign, workload-specific Aurora performance tuning, continuous validation, observable operations, and an executable reverse-CDC fallback path.

The migration completed successfully with negligible issues. Following cutover, the Aurora MySQL environment operated through approximately two months of stabilization and operational validation. Once application behavior, data integrity, performance, jobs, monitoring, and operational readiness had demonstrated sustained confidence, the legacy Oracle source system was decommissioned. The enduring lesson is straightforward: tools provide leverage, but confidence comes from engineering evidence - and modernization is complete only when the legacy platform can be retired with confidence.

BEYONDx COMMITMENT	A migration ends at cutover. A modernization ends when the legacy platform can be retired with confidence.
-------------------------------	---

Appendix A - Pre-Cutover and Stabilization Checklist

- Target objects and production-derived mappings accounted for and approved.
- DEV and PreProd DMS migrations completed with prerequisite signoff.
- Nested-table flattening and application dependencies validated.
- Functional-index replacements and target access paths reviewed.
- Partition boundaries, composite keys, business-invariant controls, and trigger decomposition validated.
- Large-table and LOB migrations complete or within approved CDC convergence plan.
- Forward CDC latency within cutover threshold; reverse CDC, binlog configuration, retention, and fallback validated.
- CLOB/LONGTEXT forward and reverse handling validated with representative payloads.
- Application Java jobs and Aurora maintenance/archive jobs tested; audit/control metrics operational.
- EST temporal behavior validated against application requirements.
- High-CPU SQL, long-running transactions, composite indexes, deadlocks, and transaction isolation tested under representative load.
- Aurora parameter changes, temporary-table capacity, slow-query capture, and concurrency settings validated.
- Post-cutover index-governance process established for redundant and potentially unused indexes.
- Backups, monitoring, maintenance, support ownership, and fallback decision points confirmed.

Appendix B - Representative Operational Queries

```
SELECT * FROM sys.schema_redundant_indexes;  
SELECT * FROM sys.schema_unused_indexes;  
SELECT * FROM sys.schema_index_statistics;
```

These views are evidence sources. Removal decisions require representative workload history, business-cycle review, application-owner validation, and controlled change management.

Appendix C - AI Agent Review Prompts for Migration Teams

- **Discovery:** What database objects, dependencies, jobs, interfaces, or business rules have not yet been accounted for?
- **Conversion:** Which source behaviors have no direct Aurora MySQL equivalent, and what capability must the replacement preserve?
- **DMS:** Which tables create the greatest restart, LOB, CDC, or cutover risk, and how can tasks be decomposed?
- **Validation:** What evidence would prove structural, data, functional, temporal, and operational equivalence?
- **Performance:** Which high-CPU or long-running statements are symptoms of missing access paths, transaction design, or parameter constraints?
- **Fallback:** What target-side changes would be lost if rollback were required now, and has the reverse path been tested?
- **Retirement:** What residual connections, jobs, reports, interfaces, or compliance dependencies still require the source platform?

Appendix D - Selected Technical References



Amazon Aurora - Aurora MySQL configuration parameters:

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Reference.ParameterGroups.html>

AWS Database Migration Service - Best practices and parallel full load:

https://docs.aws.amazon.com/dms/latest/userguide/CHAP_BestPractices.html

AWS Database Migration Service - LOB support: https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Tasks.LOBSupport.html

AWS Database Migration Service - MySQL-compatible source prerequisites and limitations:

https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Source.MySQL.html

MySQL 8.0 Reference Manual - sys schema object index and unused-index views:

<https://dev.mysql.com/doc/refman/8.0/en/sys-schema-object-index.html>

About BeyondX LLC

BeyondX LLC focuses on enterprise modernization across legacy data platforms, relational databases, cloud data services, migration engineering, and production-readiness disciplines. BeyondX emphasizes discovery, validation, engineering judgment, and practical modernization paths that allow organizations to retire legacy platforms with confidence.

Website: beyondxllc.com

Document: BeyondX TP-010 V6 - Large-Scale Oracle 19c on Exadata to Amazon Aurora MySQL 8.0 Modernization

16. Independent Data Migration Audit and Fast Validation

At approximately 50 TB, validation must be rigorous without becoming an avoidable extension of the migration window. AWS DMS validation can provide useful evidence, but full validation at very large scale can add elapsed time and workload. The modernization therefore benefits from an independent, repeatable audit layer driven by external Python scripts that query Oracle 19c and Aurora MySQL directly and compare targeted metrics.

Fast screening first; deep validation where evidence requires it. The audit framework can compare exact or statistics-based row counts, table and partition sizes, LOB volumes, key/date ranges, NULL counts, business aggregates, and selected checksums. During active CDC, comparisons must be taken at a consistent logical checkpoint so normal transaction activity is not mistaken for migration loss.

16.1 Oracle 19c quick-validation queries

```
-- Exact row count (use selectively on very large tables)
SELECT COUNT(*) AS row_count FROM owner.table_name;

-- Fast statistics-based row estimate
SELECT owner, table_name, num_rows, last_analyzed
FROM dba_tables
WHERE owner='OWNER' AND table_name='TABLE_NAME';

-- Segment size
SELECT owner, segment_name,
       ROUND(SUM(bytes)/POWER(1024,3),2) AS size_gb
FROM dba_segments
WHERE owner='OWNER' AND segment_name='TABLE_NAME'
GROUP BY owner, segment_name;

-- CLOB volume
SELECT COUNT(*) AS row_count,
       ROUND(SUM(DBMS_LOB.GETLENGTH(clob_column))/POWER(1024,3),2) AS clob_gb
FROM owner.table_name;
```

16.2 Aurora MySQL quick-validation queries

```
-- Exact row count
SELECT COUNT(*) AS row_count FROM schema_name.table_name;

-- Fast table estimate and physical size
SELECT table_schema, table_name, table_rows,
       ROUND(data_length/POWER(1024,3),2) AS data_gb,
       ROUND(index_length/POWER(1024,3),2) AS index_gb,
       ROUND((data_length+index_length)/POWER(1024,3),2) AS total_gb
FROM information_schema.tables
WHERE table_schema='schema_name' AND table_name='table_name';

-- LONGTEXT volume
SELECT COUNT(*) AS row_count,
       ROUND(SUM(OCTET_LENGTH(longtext_column))/POWER(1024,3),2) AS longtext_gb
FROM schema_name.table_name;
```

16.3 Partition and business-range reconciliation

For very large monthly partitioned tables, period-level comparison is often more actionable than one giant count. A mismatch can immediately be localized to a month or partition rather than an entire multi-billion-row object.

```
-- Oracle monthly reconciliation
SELECT TO_CHAR(crte_dt,'YYYY-MM') AS month_year, COUNT(*) AS row_count
FROM owner.table_name
GROUP BY TO_CHAR(crte_dt,'YYYY-MM')
ORDER BY month_year;

-- Aurora MySQL monthly reconciliation
SELECT DATE_FORMAT(crte_dt,'%Y-%m') AS month_year, COUNT(*) AS row_count
FROM schema_name.table_name
GROUP BY DATE_FORMAT(crte_dt,'%Y-%m')
ORDER BY month_year;

-- Comparable business-range checks (run on both platforms with native syntax)
SELECT COUNT(*) AS row_count,
       MIN(aplctn_seq_nbr) AS min_key, MAX(aplctn_seq_nbr) AS max_key,
       MIN(crte_dt) AS min_crte_dt, MAX(crte_dt) AS max_crte_dt
FROM table_name;
```

Audit metric	Purpose	Typical treatment
Row count	Completeness screening	Exact for manageable tables; statistics/partition strategy for very large tables
Table / partition size	Volume comparison	Directional evidence; physical storage differs by engine
LOB bytes	CLOB/BLOB completeness	Compare payload volume plus targeted content checks
PK/date ranges	Boundary coverage	Fast detection of missing ranges
NULL / aggregates	Semantic sanity	Use business-critical columns
Exceptions	Focused investigation	Persist source/target differences with PASS/FAIL/INVESTIGATE

A Python audit orchestrator can execute paired source/target SQL, normalize the results, calculate differences and percentages, and persist evidence such as audit_run_id, environment, schema, table, metric, source_value, target_value, difference, status, timestamps, and comments. ChatGPT/AI agents can help generate paired queries and comparison logic; the measurements themselves must come from the databases.

BEYONDX ENGINEERING PRINCIPLE: Migration validation should be independent, repeatable, measurable, and fast enough to run frequently.

17. Oracle Sequence Modernization and Key Generation

Oracle 19c supports independent sequence objects that may be called by applications, triggers, procedures, jobs, or direct SQL. Aurora MySQL 8.0 does not provide an Oracle-style standalone sequence object. Each sequence therefore requires dependency discovery and a deliberate target pattern rather than a mechanical syntax replacement.

17.1 Inventory Oracle sequences and consumers

```
SELECT sequence_owner, sequence_name, min_value, max_value,
       increment_by, cycle_flag, order_flag, cache_size, last_number
FROM dba_sequences
WHERE sequence_owner='OWNER'
ORDER BY sequence_name;

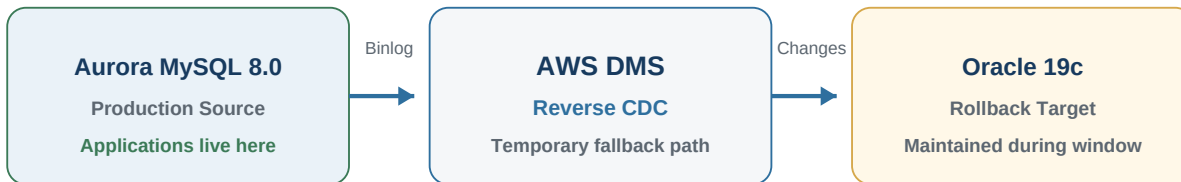
SELECT owner, name, type, referenced_owner, referenced_name, referenced_type
FROM dba_dependencies
WHERE referenced_type='SEQUENCE' AND referenced_owner='OWNER'
ORDER BY referenced_name, owner, name;

-- Locate NEXTVAL / CURRVAL references in stored source
SELECT owner, name, type, line, text
FROM dba_source
WHERE owner='OWNER'
      AND (UPPER(text) LIKE '%.NEXTVAL%' OR UPPER(text) LIKE '%.CURRVAL%')
```

ORDER BY owner, name, type, line;

Reference Architecture - Cutover and Reverse CDC Fallback

After the controlled cutover, Aurora becomes the production source. A temporary reverse-CDC path preserves an executable rollback option while the target platform stabilizes.



Fallback engineering controls

Aurora CDC source	ROW binary logging + FULL row image; retention engineered for fallback window
LOB reverse path	LONGTEXT -> Oracle CLOB with DMS LOB settings sized for required payload
Operational gate	Forward migration reconciled; applications quiesced for controlled cutover
Rollback decision	If fallback is invoked: stop/coordinate writes, reconcile reverse CDC, redirect applications
Retirement	Reverse path removed after stabilization and confidence in Aurora operation

CUTOVER AND FALLBACK LOGIC

Quiesce Oracle writes -> confirm forward CDC within the agreed cutover threshold -> final reconciliation -> redirect applications to Aurora -> verify reverse CDC. If rollback is invoked, coordinate writes, reconcile the reverse path, and redirect applications under the approved runbook.

17.2 Target key-generation patterns

Oracle usage	Aurora MySQL modernization pattern	Engineering consideration
Sequence used for one generated key	AUTO_INCREMENT where application behavior permits	Initialize above migrated maximum; validate insert behavior
Application does not want identity behavior	Application or controlled database routine generates value	Define ownership, retries, and concurrency semantics
Multiple independent counters	Dedicated sequence-control table / routine	Counters remain independent and atomic
Sequence shared across tables	Central sequence-control mechanism	Preserve global uniqueness requirement
Business/document number	Do not assume AUTO_INCREMENT	Gaps, ordering and restart semantics may be business-significant

```
CREATE TABLE sequence_control (
  sequence_name VARCHAR(128) PRIMARY KEY,
  current_value BIGINT NOT NULL,
  increment_by BIGINT NOT NULL DEFAULT 1,
  updated_at DATETIME NOT NULL
);
```

```
-- Example atomic increment pattern; validate under project concurrency
UPDATE sequence_control
SET current_value = LAST_INSERT_ID(current_value + increment_by),
    updated_at = NOW()
WHERE sequence_name='APPLICATION_SEQ';
SELECT LAST_INSERT_ID() AS next_value;
```

Before conversion, determine who calls each sequence, whether gaps are acceptable, whether ordering matters, whether the value is a surrogate key or business reference, whether it is shared across tables, and what current value must be preserved at cutover.

BEYONDX ENGINEERING PRINCIPLE: Preserve key-generation semantics, not the Oracle sequence syntax.

18. Oracle 19c to Aurora MySQL 8.0 Mapping Reference

Datatype mapping must consider business meaning, range, precision, character semantics, timezone behavior, and actual stored values. Oracle DATE is the classic example: it stores both date and time to the second but does not itself carry timezone information. A target type should therefore be selected from the column's temporal semantics, not simply its source datatype name.

Oracle 19c	Aurora MySQL 8.0	Guidance
NUMBER(p,0)	TINYINT / SMALLINT / INT / BIGINT / DECIMAL	Choose from actual range and precision
NUMBER(p,s)	DECIMAL(p,s)	Preserve required precision and scale
VARCHAR2 / CHAR	VARCHAR / CHAR	Review byte-vs-character semantics and charset
CLOB	LONGTEXT	Profile maximum LOB sizes and DMS mode
BLOB	LONGBLOB	Validate payload size and application access
RAW	VARBINARY / BINARY	Choose by variable/fixed semantics
DATE - calendar date only	DATE	Examples: DOB or effective date when time is not meaningful
DATE - literal wall-clock date/time	DATETIME	Preserves literal value without automatic timezone conversion
DATE - true event instant	TIMESTAMP	Use when timezone/session conversion is intentionally designed
TIMESTAMP	DATETIME(6) or TIMESTAMP(6)	Choose from required temporal semantics

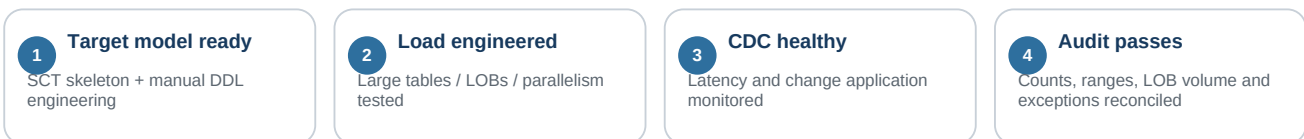
Oracle 19c	Aurora MySQL 8.0	Guidance
SEQUENCE	AUTO_INCREMENT or controlled generator	Discover dependencies before redesign

Reference Architecture - Forward Migration (Full Load + CDC)

Applications remain on Oracle while AWS DMS establishes the Aurora copy and continuously applies captured changes. Target schema engineering and independent validation occur in parallel.



Engineering gates before cutover



BEYONDX ENGINEERING PRINCIPLE

Full Load moves the baseline. CDC preserves continuity. Independent audit provides the evidence required to authorize cutover.

18.1 Temporal discovery before mapping Oracle DATE

```
-- Inventory Oracle temporal columns
SELECT owner, table_name, column_name, data_type,
       data_length, data_precision, data_scale
FROM dba_tab_columns
WHERE owner='OWNER'
      AND data_type IN ('DATE','TIMESTAMP','TIMESTAMP WITH TIME ZONE',
                        'TIMESTAMP WITH LOCAL TIME ZONE')
ORDER BY table_name, column_id;

-- Determine whether an Oracle DATE actually contains time-of-day values
SELECT COUNT(*) AS total_rows,
       SUM(CASE WHEN date_column <> TRUNC(date_column) THEN 1 ELSE 0 END)
       AS rows_with_time
FROM owner.table_name;

-- Target temporal inventory
SELECT table_schema, table_name, column_name, data_type, column_type,
       datetime_precision
FROM information_schema.columns
WHERE table_schema='schema_name'
      AND data_type IN ('date','datetime','timestamp')
ORDER BY table_name, ordinal_position;
```

For this modernization, application temporal requirements governed the implementation. The paper therefore treats TIMESTAMP as a design option rather than a universal replacement for Oracle DATE. Historical ranges, sentinel values, DST behavior, JDBC/session timezone behavior, and application expectations must all be profiled and tested before changing temporal semantics.

BEYONDX ENGINEERING PRINCIPLE: Do not map Oracle DATE by datatype name. Map it by business temporal semantics.

19. Database-Application Integration and Coordination

Application modernization was performed through coordination with the application teams rather than by treating database engineers as application developers. Application source was maintained in Git-based repositories, while the database modernization team identified database changes that could affect application behavior and communicated those impacts for implementation and testing by the responsible application teams.

Database change	Application impact to communicate
Datatype / temporal mapping	Entity/DTO types, serialization, date/time assumptions, validation
Sequence / identity redesign	Generated-key behavior and insert paths
PK/FK / partition redesign	Key assumptions, relationship enforcement, query predicates
Trigger / stored logic changes	Behavior previously executed inside Oracle
Oracle jobs moved to Java	Scheduling, retries, observability, ownership
SQL dialect differences	Native SQL, Oracle functions, hints, pagination and procedure calls
Performance/index changes	Representative application workload and regression testing

AI agents can assist the database team by turning each database change into an application-impact checklist: search the Git repository for Oracle-specific SQL, sequence calls, stored-procedure references, column assumptions, timezone dependencies, and transaction-sensitive paths. Application engineers remain responsible for implementation and application-level signoff.

BEYONDX ENGINEERING PRINCIPLE: Modernization responsibilities may be divided across teams, but dependencies cannot be.

V6 Reference Notes

The SQL examples in this paper are reference patterns and must be adapted to schema names, privileges, partition strategy, engine minor version, workload, and security controls. Statistics-based row counts and physical-size comparisons are screening evidence, not substitutes for business reconciliation. Sequence-emulation routines require concurrency testing. Temporal mappings require explicit application semantics and range testing.

V6 additions: independent Python-driven migration audit and fast validation; paired Oracle/Aurora validation SQL; partition-level reconciliation; Oracle sequence discovery and target key-generation patterns; datatype/behavior mapping reference; Oracle DATE decision guidance; and database-application integration coordination.

These additions reinforce the paper's central position: modernization is successful only when data, behavior, performance, operations, and application dependencies are all proven on the target platform - and the legacy platform can be retired with confidence.